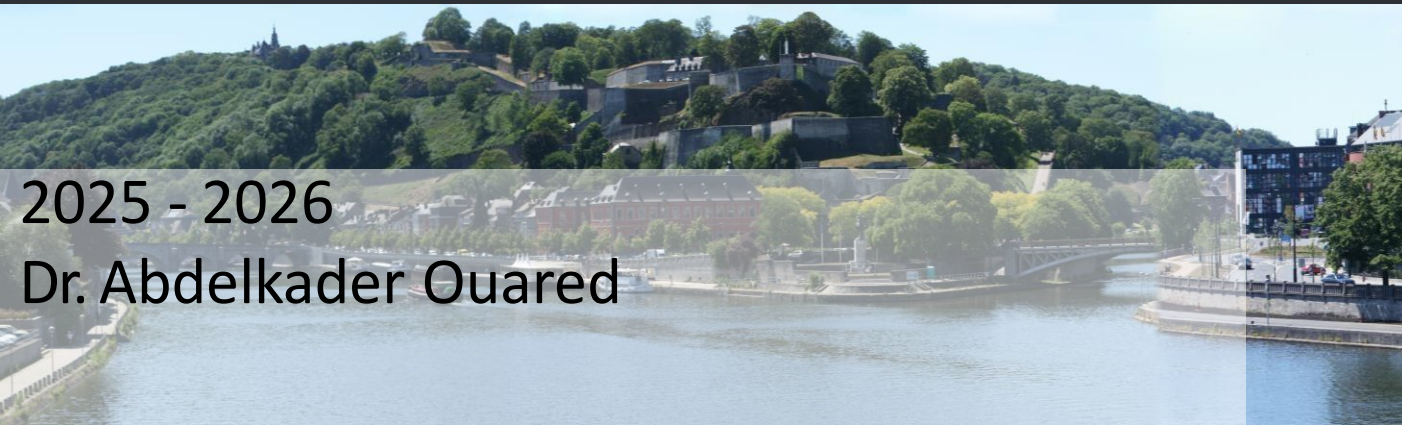


Programmation Mobile

SQLite et la bibliothèque de persistance Room



2025 - 2026
Dr. Abdelkader Ouared



Agenda

Introduction

ORM

Room & SQLite

Composantes
de Room

Implémentation
de Room

Exercice

Introduction



Base de données locale

- Application mobile en mode hors ligne (pas de connexion ; on doit utiliser une base de données locale)
- Expérience utilisateur : les applications de réseaux sociaux chargent les derniers posts hors ligne.
- Vente de produits pharmaceutiques (test immédiat si le médicament est non remboursable).
- Autres cas d'usage

Exemple de BD locale pour les applications mobiles

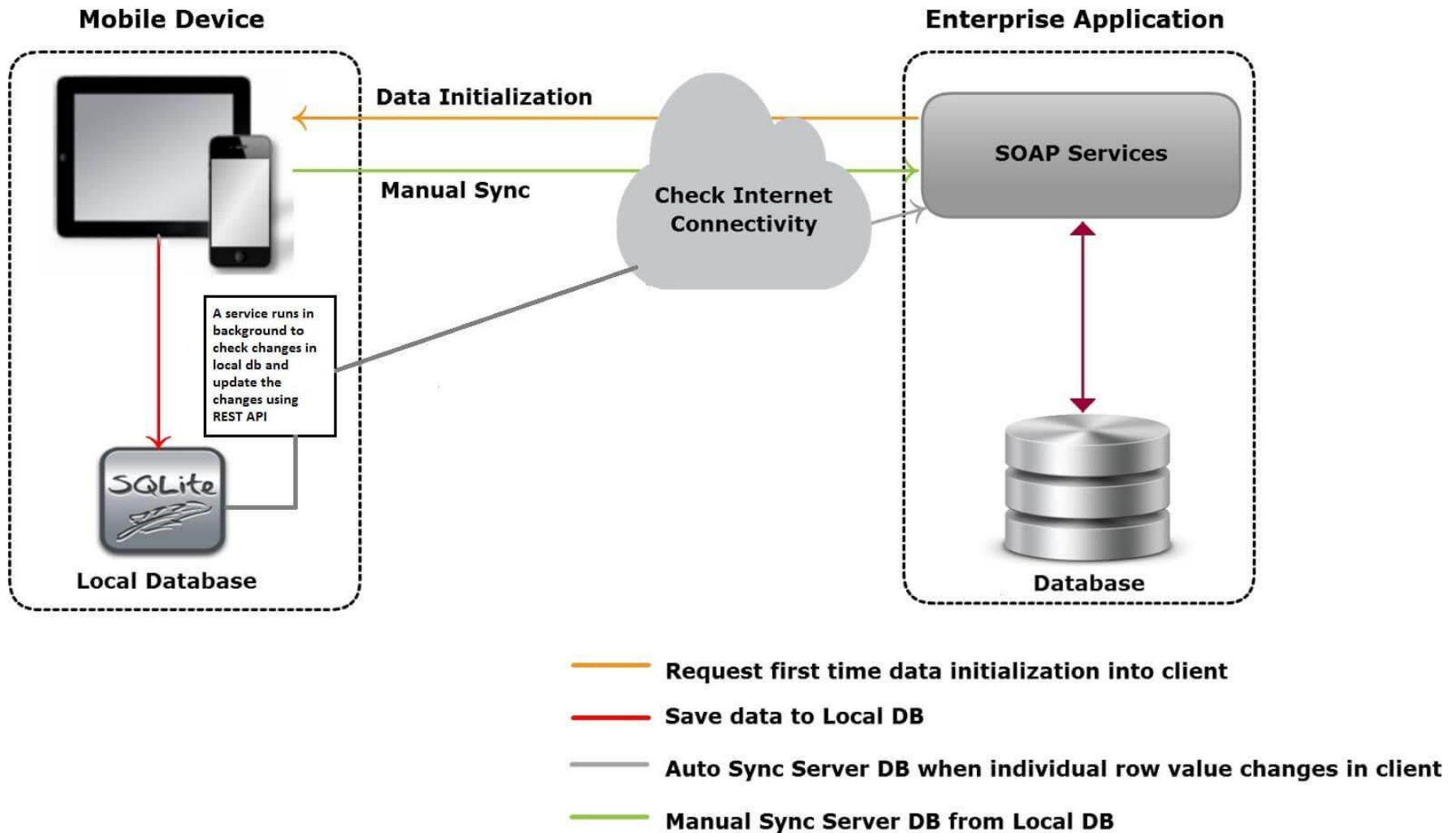


Firestore Realtime Database (local cache activé)

Solutions de bases de données locales pour applications mobiles

Solution	Type	Langage/Plateforme
SQLite	Relationnelle	Android, Flutter, iOS
Room	ORM (SQLite)	Android (Kotlin, Java)
Realm	NoSQL	Android, Flutter, iOS
Hive	Clé-valeur	Flutter
ObjectBox	ORM (NoSQL)	Android, Flutter, iOS
Firebase	Cloud/NoSQL	Android, Flutter, iOS
SharedPreferences / SecureStorage	Clé-valeur	Android, Flutter

Base de données locale



Base de données dans une application mobile

1. Base de données locale

- **Base de données SQLite** dans les applications Android et iOS
- **SQLiteOpenHelper**: fournit un ensemble de méthodes pour gérer une BD SQLite locale.

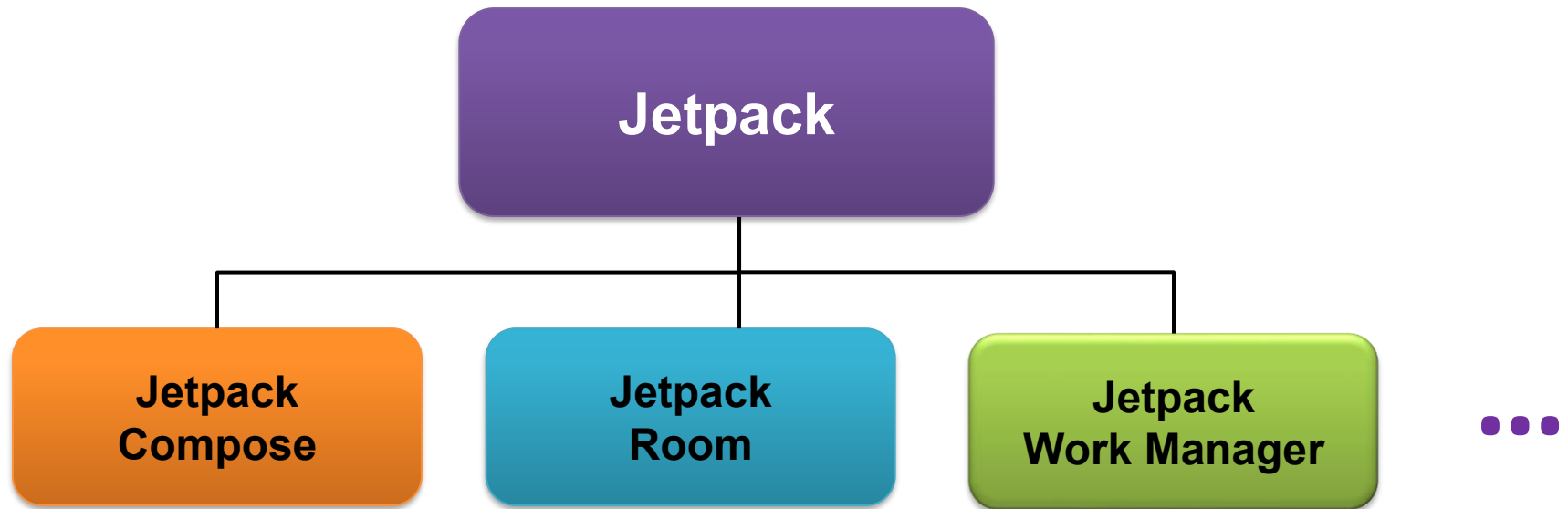
➤ **Surcharge** 😞

- **ORM**: par ex. **ROOM (Jetpack Room library)**

2. Base de données distante (or serveur de BD," ou « BD Cloud)

- **SGBD**: MySQL, PostgreSQL, MongoDB, Firebase Realtime Database, et autres.
- **API Webserver**
- **ORM**: par ex. Java Hibernate, PHP Doctrine

Bibliothèques Android **Jetpack**

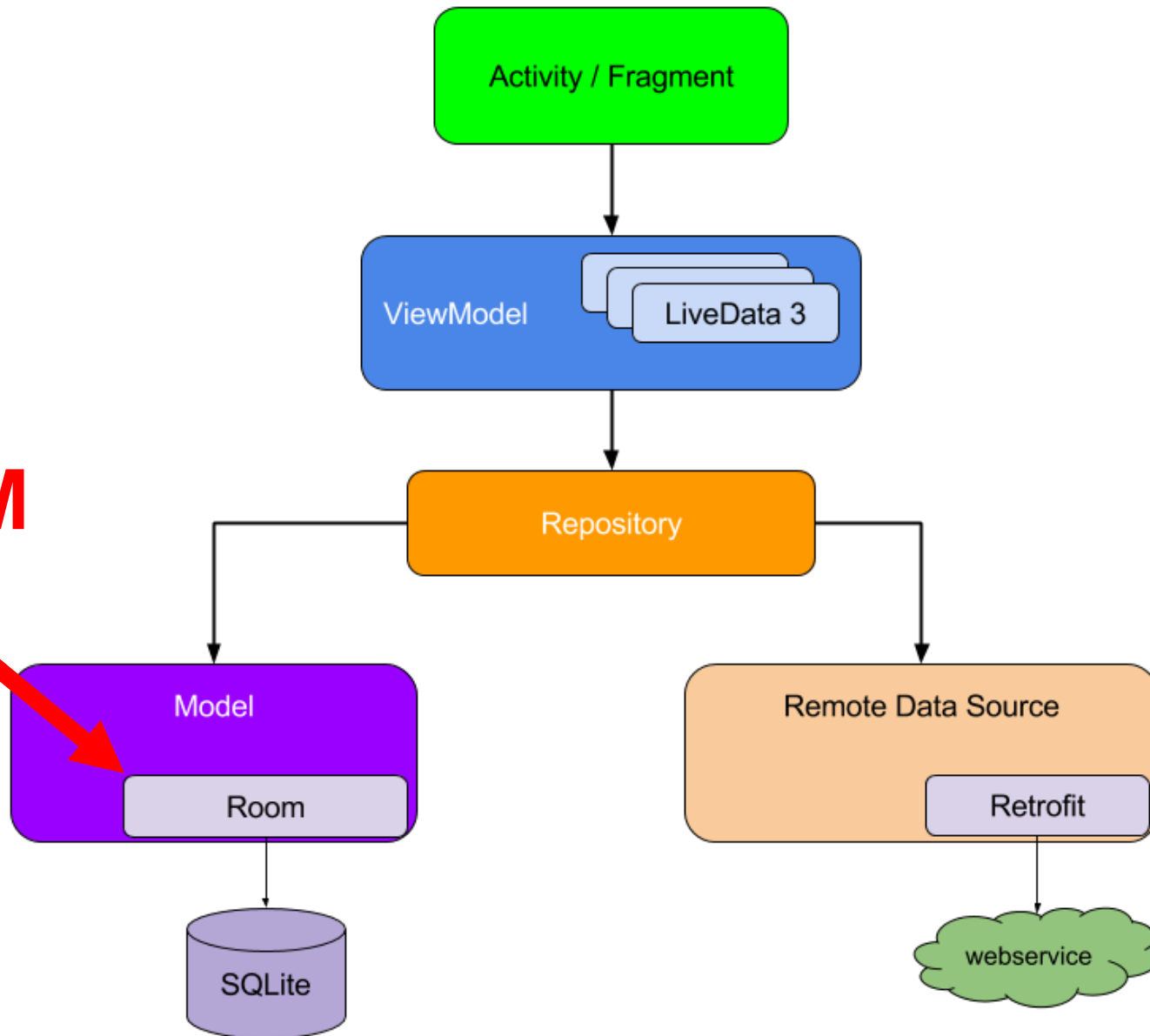


- Ajoutez leurs dépendances au fichier **build.gradle** de votre application.

```
implementation "androidx.compose.ui:ui:1.0.0"  
implementation "androidx.room:room-runtime:2.3.0"  
implementation "androidx.work:work-runtime:2.7.0"
```

Android Jetpack Room

ORM



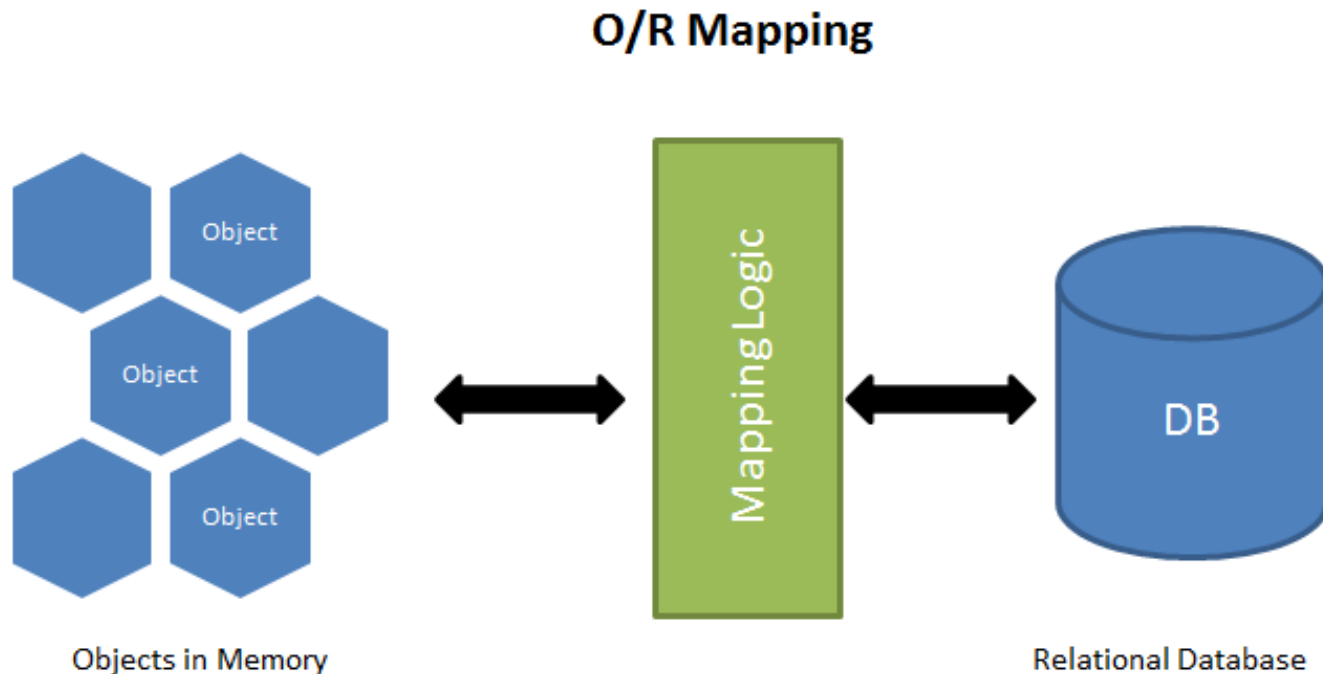
ORM - Mapping Objet-Relationnel



Object Relational Mapping (ORM)

Mappage relationnel objet (ORM) ▪ Utiliser une couche de mappage pour mapper entre les objets et les tables

- Mappage d'une représentation de données d'un modèle objet à un modèle de données relationnel avec un schéma basé sur SQL



Avantages de l'ORM

- ☐ Interaction simplifiée avec la base de données
- ☐ Réutilisabilité du code
- ☐ Productivité
- ☐ Indépendance à la base de données
- ☐ Abstraction des requêtes complexes
- ☐ Meilleure maintenabilité
- ☐ Création et migration automatiques de tables
- ☐ Contrôle de la concurrence

Inconvénients de l'ORM

- Perte de contrôle sur le processus de persistance
- Peut être plus difficile d'ajuster les requêtes
- Les caractéristiques de performance de l'outil peuvent affecter les performances de votre application

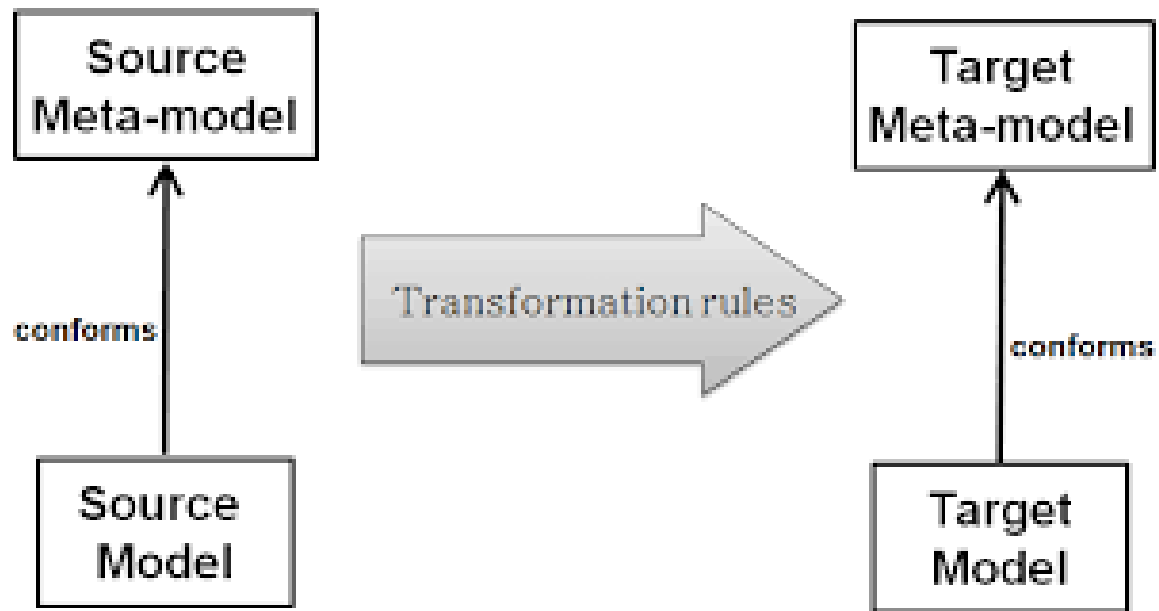
Cas des base de données NoSQL

- ❑ Dans le cas de NoSQL (par ex. MongoDB, Cassandra, Redis, etc.) nous utiliserons un **ODM** (**Object Document Mapping**) pour obtenir les mêmes résultats.
- C'est comme un ORM pour les bases de données NoSQL (bases de données de documents, bases de données graphe, etc.)."

Note: In document-oriented NoSQL databases, documents are stored in JSON format in the collection (a table in SQL databases is like a collection in NoSQL databases).

Question

- Comment concevoir et implémenter un ORM ??



Cas des base de données Graphe

UMLtoGraphDB: Mapping Conceptual Schemas to Graph Databases

Gwendal Daniel¹(✉), Gerson Sunyé¹, and Jordi Cabot^{2,3}

¹ AtlanMod Team, Inria, Mines Nantes, Lina, Nantes, France
`{gwendal.daniel,gerson.sunye}@inria.fr`

² ICREA, Barcelona, Spain

³ Internet Interdisciplinary Institute, UOC, Barcelona, Spain
`jordi.cabot@icrea.cat`

Note: In document-oriented NoSQL databases, documents are stored in JSON format in the collection (a table in SQL databases is like a collection in NoSQL databases).

Exemple de bibliothèque ODM

- "Une bibliothèque ODM populaire est **Mongoose**.
- **Mongoose** est une bibliothèque JavaScript qui offre une solution simple basée sur un schéma pour modéliser les données de votre application.
 - Elle inclut une conversion de type intégrée, une validation, la construction de requêtes et prend en charge les middleware.
 - Elle est largement utilisée avec Node.js et peut être intégrée dans des applications mobiles construites avec des frameworks tels que React Native ou Ionic."

SQLite & ROOM



SQLite

- C'est une base de données légère
- Bénéfique pour les applications hors ligne
- Nécessite très peu de mémoire
- Une base de données gérée automatiquement
- Open source et gratuit

SQLite a été développé par D. Richard Hipp. Elle a été créée en août 2000.

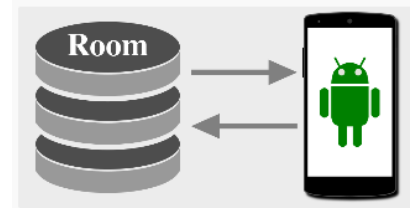
SQLite

SQLite	SQL
Is a relational database management system which is written in ANSI-C	Is structured query language used to query relational database system. It is written in C language
Is file based	Is server based
Does not support stored procedures (serverless)	Support stored procedures
Is not client-server architecture	Client-server architecture
Embedded in client	Embedded in server
Small size (light)	Large size (heavy)
No configurations	A lot of configurations

Room



Room



- ❑ Room est une bibliothèque de persistance faisant partie d' [Android Jetpack](#).
- ❑ Room fournit une couche d'abstraction sur SQLite pour permettre un accès fluide aux bases de données tout en exploitant toute la puissance de SQLite.
- ❑ Room est désormais considéré comme une meilleure approche pour la persistance des données que **SQLiteDatabase**. Il facilite l'utilisation des objets **SQLiteDatabase** dans votre application, en réduisant la quantité de code passe-partout et en vérifiant les requêtes SQL au moment de la compilation.

Pourquoi on utilise Room?

- ❑ Vérification au moment de la **compilation** des requêtes SQL. chaque **@Query** et **@Entity** sont vérifiés au moment de la compilation, ce qui préserve votre application **des problèmes de crash** au moment de l'exécution et vérifie non seulement la seule syntaxe, mais également les tables manquantes.
- ❑ Facilement intégré à d'autres composants d'architecture (comme **LiveData**)
- ❑ Faciliter le travail des développeurs avec des bases de données dans leurs applications Android.
- ❑ Une réduction du code passe-partout et de meilleures performances par rapport au **SQLiteOpenHelper** traditionnel.

Bibliothèque de persistance

Room vs SQLite

- ❑ Dans le cas de SQLite, **il n'y a pas de vérification au moment de la compilation** des requêtes SQLite brutes. Mais dans Room, il y a une validation SQL au moment de la compilation.

SQLite brut

```
db.rawQuery("SELECT namee FROM users", null) // typo "namee"
```

Room

```
kotlin

@Dao
interface UserDao {
    @Query("SELECT namee FROM users")
    fun getAllNames(): List<String>
}
```

Bibliothèque de persistance

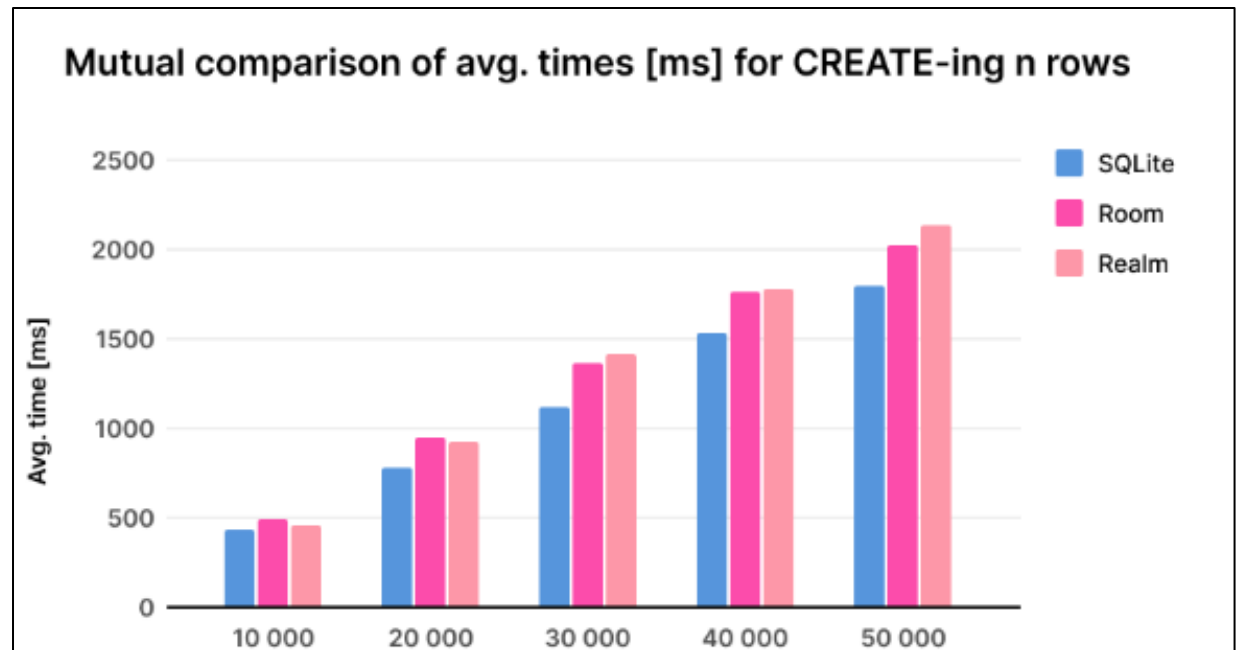
Room vs SQLite

- ❑ Dans le cas de SQLite, **il n'y a pas de vérification au moment de la compilation** des requêtes SQLite brutes. Mais dans Room, il y a une validation SQL au moment de la compilation.
- ❑ Vous devez utiliser **beaucoup de code** passe-partout pour convertir entre les requêtes SQL et les objets de données Java.
- ❑ Mais Room mappe nos objets de base de données sur un objet Java sans code passe-partout.
- ❑ À mesure que votre schéma change, vous devez **mettre à jour manuellement** les requêtes SQL concernées. Room traite ce problème.
- ❑ Room est conçu pour fonctionner avec **LiveData** et **RxJava** pour l'observation des données, contrairement à SQLite.

Bibliothèque de persistance Room vs SQLite

Annotation Processing

- Java Annotation Processing (APT), KAPT: (Kotlin Annotation Processing Tool)
 - Build Time ☹️



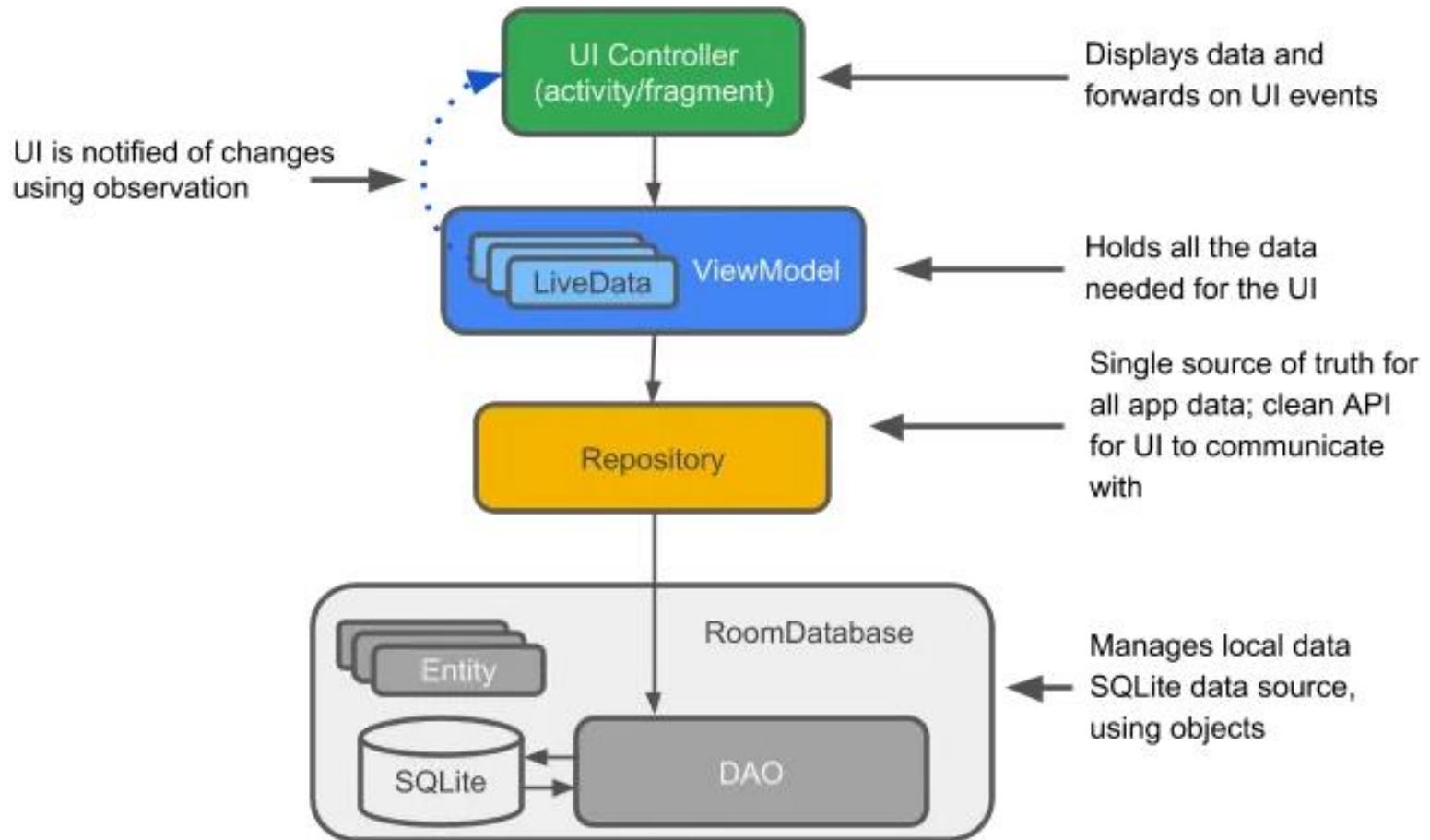
- KSP: Kotlin Symbol Processing [*]

[*] <https://kotlinlang.org/docs/ksp-overview.html>

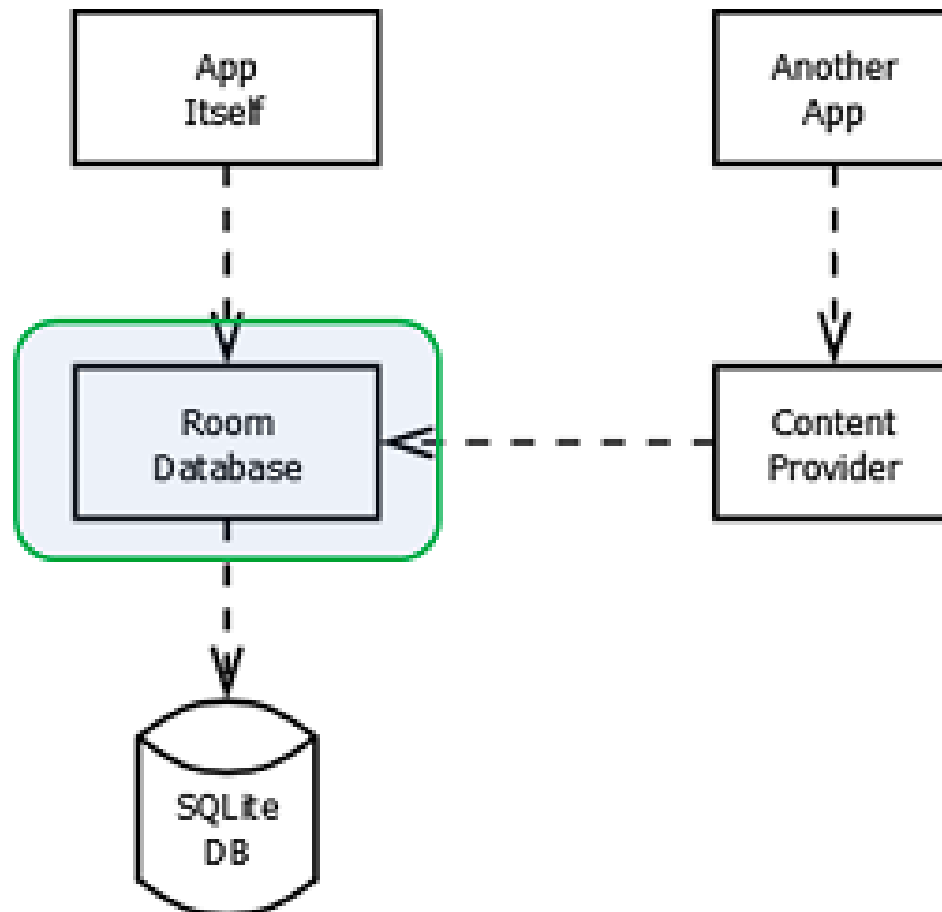
Les principaux composants de Room Database ?

- ❑ **Entity:** Représente une table dans la base de données et est défini comme une classe de données annotée avec l'annotation **@Entity**. Chaque propriété de la classe de données correspond à une colonne du tableau.
- ❑ **DAO (Data Access Object):** Contient des méthodes pour définir des opérations de base de données (CRUD) pour les entités. Les DAO sont des interfaces annotées avec **@Dao**, et les méthodes à l'intérieur du DAO sont annotées avec des requêtes SQL utilisant des annotations telles que **@Insert**, **@Update**, **@Delete**, etc.
- ❑ **Database:** Représente la base de données elle-même et est définie comme une classe abstraite annotée avec **@Database**. Il comprend une liste d'entités et un numéro de version. Room génère l'implémentation de cette classe au moment de la compilation.

Les principaux composants de Room Database ?



Room et fournisseur de contenu (Content provider)



Principaux composants de Room DB



Room comporte trois composants principaux de Room DB :

- Entity
- Dao
- Database

Entity

Représente une table dans la base de données. Room crée une table pour chaque classe comportant une annotation **@Entity**, les champs de la classe correspondent aux colonnes de la table. Par conséquent, les classes d'entités ont tendance à être de petites classes de modèles qui ne contiennent aucune logique.

```
@Entity(tableName = "user")  
data class User(  
    @PrimaryKey val id: Int,  
    val name: String  
)
```

Entity: Les annotations utiles

@Entity — chaque classe de modèle avec cette annotation aura une table de mappage dans DB

- **foreignKeys** — noms de clés étrangères
- **indices** — liste des indexes sur la table
- **primaryKeys** — noms des clés primaires des entités
 - **autoGenerate** — s'il est défini sur true, SQLite générera un identifiant unique pour la colonne
 - **@PrimaryKey(autoGenerate = true)**
- **tableName** — noms de la table
- **@ColumnInfo**: Spécifie les détails d'une colonne, tels que son nom dans la table, si elle est indexée ou si elle autorise les valeurs nulles.
 - **@ColumnInfo(name = "column_name")**

Entity: Les annotations utiles

@Ignore — le champ ne sera pas persisté par Room

@Embedded — les champs imbriqués peuvent être référencés directement dans les requêtes SQL.

@Relation: L'annotation est utilisée pour définir les relations entre les entités dans Room. Il vous permet de récupérer des entités associées avec une seule requête, simplifiant ainsi les requêtes de bases de données complexes.

Exemple de code

```
@Entity(tableName = "book")
data class Book(
    @PrimaryKey val id: Int,
    val title: String,
    @ColumnInfo(name = "user_id") val userId: Int
)
data class UserWithBooks(
    @Embedded val user: User,
    @Relation(
        parentColumn = "id",
        entityColumn = "user_id"
    )
    val books: List<Book>
)
@Dao
interface UserDao {
    @Transaction
    @Query("SELECT * FROM user")
    fun getUsersWithBooks(): List<UserWithBooks>
}
```

DAO: Data Access Object

Les DAO sont chargés de définir les méthodes qui accèdent à la base de données.

Dans le SQLite initial, nous utilisons les objets **Cursor**. Avec Room, nous n'avons pas besoin de tout le code lié au curseur et pouvons simplement définir nos requêtes à l'aide d'annotations dans la classe DAO.

Exemple de code

```
@Entity(tableName = "user")
data class User(
    @PrimaryKey val id: Int,
    val name: String,
    val age: Int
)
```

```
@Dao
interface UserDao {
    @Insert
    fun insert(user: User)
    @Update
    fun update(user: User)
    @Delete
    fun delete(user: User)
    @Query("SELECT * FROM user")
    fun getAllUsers(): List<User>
}

@Database(entities = [User::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
}
```

Database

Contient le support de base de données et sert de point d'accès principal pour la connexion sous-jacente aux données relationnelles persistantes de votre application.

Pour créer une base de données, nous devons définir une classe abstraite qui étend **RoomDatabase**. Cette classe est annotée avec **@Database**, répertorie les entités contenues dans la base de données, et les DAO qui y accèdent. La classe annotée avec **@Database** doit remplir les conditions suivantes :

- Soyez une classe abstraite qui étend [RoomDatabase](#).
- Incluez la liste des entités associées à la base de données dans l'annotation.
- Contient une méthode abstraite qui n'a aucun argument et renvoie la classe annotée avec [@Dao](#)

Implémentation de Room



Step 1: Add the Gradle dependencies

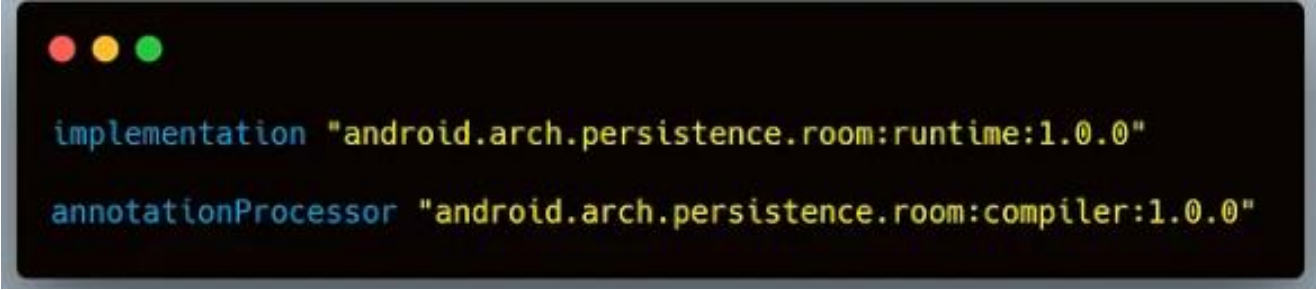
To add it to your project, open the project level `build.gradle` file and add the highlighted line as shown below:



```
allprojects {  
    repositories {  
        jcenter()  
        maven { url 'https://maven.google.com' }  
    }  
}
```

Step 1: Add the Gradle dependencies

Open the build.gradle file for your app or module and add dependencies:

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains two lines of Gradle Kotlin DSL code. The first line is `implementation "android.arch.persistence.room:runtime:1.0.0"` and the second line is `annotationProcessor "android.arch.persistence.room:compiler:1.0.0"`.

```
implementation "android.arch.persistence.room:runtime:1.0.0"  
annotationProcessor "android.arch.persistence.room:compiler:1.0.0"
```

Section 1: Setting Up Room Database

Step 1: Add Dependencies

Open your app's **build.gradle** module level file and add the necessary dependencies for Room and Kotlin Coroutines (for asynchronous operations):

```
gradle
dependencies {
    def roomVersion = "2.4.0" // Check for the latest version
    implementation "androidx.room:room-runtime:$roomVersion"
    kapt "androidx.room:room-compiler:$roomVersion"
    implementation "androidx.room:room-ktx:$roomVersion"
    implementation "org.jetbrains.kotlinx:kotlinx-coroutines-android:1.5.2" // Add Coroutine
    dependency
}
```

Section 1: Setting Up Room Database

Ou bien

For latest android and jetpack compose if its failing with some error we might have to add ksp() {Kotlin symbol processing}.

Add below dependencies and plugin in build.gradle(Module level).

```
plugins {  
    .  
    .  
    id "com.google.devtools.ksp"  
}  
  
.  
.  
.  
  
dependencies{  
    // Room dependency  
    val room_version = "2.5.2"  
  
    implementation("androidx.room:room-ktx:$room_version")  
    // To use Kotlin annotation processing tool (kapt)  
    ksp("androidx.room:room-compiler:$room_version")  
}
```

Section 1: Setting Up Room Database

Add below class path for the ksp in build.gradle(app level).

```
plugins {  
id "com.google.devtools.ksp" version "1.8.10-1.0.9" apply false  
}
```

Step 2: Create a Model Class

Room creates a table for each class annotated with **@Entity**; the fields in the class correspond to columns in the table. Therefore, the entity classes tend to be small model classes that don't contain any logic. Our Person class represents the model for the data in the database. So let's update it to tell Room that it should create a table based on this class:

- Annotate the class with **@Entity** and use the `tableName` property to set the name of the table.
- Set the primary key by adding the **@PrimaryKey** annotation to the correct fields — in our case, this is the ID of the User.
- Set the name of the columns for the class fields using the **@ColumnInfo(name = "column_name")** annotation. Feel free to skip this step if your fields already have the correct column name.
- If multiple constructors are suitable, add the **@Ignore** annotation to tell Room which should be used and which not.

Step 2: Create a Model Class

```
import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity
data class Person(
    @PrimaryKey(autoGenerate = true)
    val id: Long = 0, // Primary key with auto-generation

    // Ignoring the 'name' attribute
    @field:Ignore
    val name: String,

    // Ignoring the 'city' attribute
    @field:Ignore
    val city: String
)
```

Step 2: Create a Model Class

```
@Entity(tableName = "person")
public class Person {
    @PrimaryKey(autoGenerate = true)
    private int id;
    @ColumnInfo(name = "name")
    private String name;
    @ColumnInfo(name = "city")
    private String city;

    public Person(int id, String name, String city) {
        this.id = id;
        this.name = name;
        this.city = city;
    }

    @Ignore
    public Person(String name, String city) {
        this.name = name;
        this.city = city;
    }
}
```


Step 3: Create Data Access Objects (DAOs)

[DAOs](#) are responsible for defining the methods that access the database. To create a DAO we need to create an interface and annotated with `@Dao` .

```
import androidx.room.Dao
import androidx.room.Insert
import androidx.room.Query

@Dao
interface PersonDao {

    @Insert
    suspend fun insertPerson(person: Person)

    @Query("SELECT * FROM Person")
    suspend fun getAllPersons(): List<Person>

    @Query("SELECT * FROM Person WHERE id = :personId")
    suspend fun getPersonById(personId: Long): Person

    // Add other methods as needed for your use case
}
```

Step 3: Create Data Access Objects (DAOs)

[DAOs](#) are responsible for defining the methods that access the database. To create a DAO we need to create an interface and annotated with `@Dao` .

```
@Dao
public interface PersonDao {
    @Query("Select * from person")
    List<Person> getPersonList();
    @Insert
    void insertPerson(Person person);
    @Update
    void updatePerson(Person person);
    @Delete
    void deletePerson(Person person);
}
```

Step 4 — Create the database

To create a database we need to define an abstract class that extends RoomDatabase. This class is annotated with @Database, lists the entities contained in the database, and the DAOs which access them.

```
@Database(entities = Person.class, exportSchema = false, version = 1)
public abstract class PersonDatabase extends RoomDatabase {
    private static final String DB_NAME = "person_db";
    private static PersonDatabase instance;

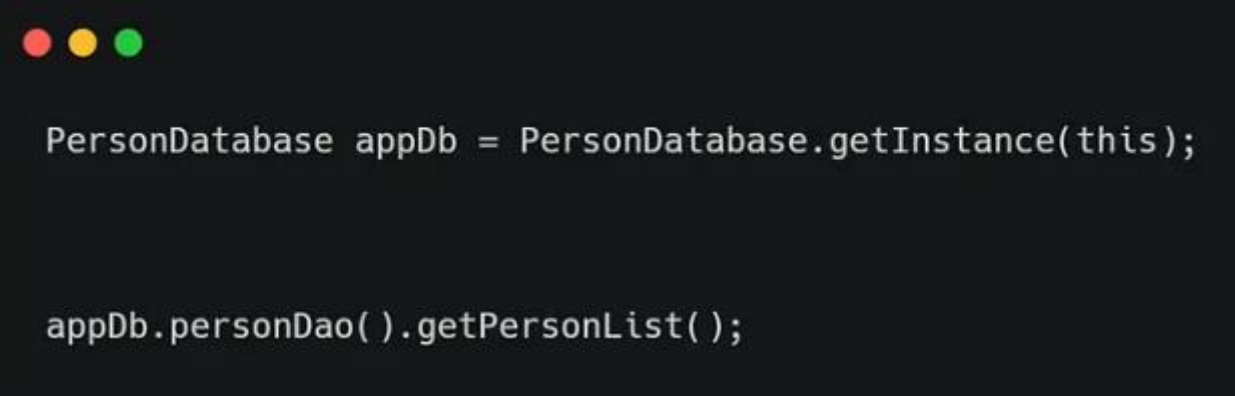
    public static synchronized PersonDatabase getInstance(Context context) {
        if (instance == null) {
            instance = Room.databaseBuilder(context.getApplicationContext(), PersonDatabase.class,
DB_NAME)
                .fallbackToDestructiveMigration()
                .build();
        }
        return instance;
    }

    public abstract PersonDao personDao();
}
```

Step 4: Managing Data

To manage the data, first of all, we need to create an instance of the database. then we can insert delete and update the database.

Make sure all the operation should execute on a different thread. In my case I use Executors



```
PersonDatabase appDb = PersonDatabase.getInstance(this);  
  
appDb.personDao( ).getPersonList( );
```

Insert:



```
PersonDatabase appDb = PersonDatabase.getInstance(this);  
Person person = new Person("Ashish", "Noida");  
appDb.personDao().insertPerson(person);
```

Delete:



```
PersonDatabase appDb = PersonDatabase.getInstance(this);  
appDb.personDao().deletePerson(person);
```

Update:



```
PersonDatabase appDb = PersonDatabase.getInstance(this);  
Person person = new Person("Ashish Rawat", "Noida");  
appDb.personDao().updatePerson(person);
```

Executing with Executor

```
AppExecutors.getInstance().diskIO().execute(new Runnable() {  
    @Override  
    public void run() {  
        final List<Person> persons = mDb.personDao().loadAllPersons();  
        mAdapter.setTasks(persons);  
    }  
});
```


Step 1: Insert Data

To insert a user into the database, you can use the `insertUser` method defined in the `UserDao`:

```
val newUser = User(username = "JohnDoe", email =  
"john@example.com")  
MyApp.database.userDao().insertUser(newUser)
```

Step 2: Retrieve Data

To retrieve all users from the database, use the `getAllUsers` method from the `UserDao`:

```
val userList: List<User> = MyApp.database.userDao().getAllUsers()
```

How does Room handle database migrations?

Room provides database migrations using the Migration class. When you update your database schema by modifying entities or DAOs, you need to create a new Migration object and add it to the RoomDatabase.Builder. Room will automatically execute the necessary migrations to keep the existing data intact.

Code Example

```
val migration1to2 = object : Migration(1, 2) {  
    override fun migrate(database: SupportSQLiteDatabase) {  
        // Migration code from version 1 to 2  
    }  
}
```

```
val appDatabase = Room.databaseBuilder(context, AppDatabase::class.java,  
    "app_database")  
    .addMigrations(migration1to2)  
    .build()
```

How can you observe database changes using LiveData with Room?

Room provides the LiveData class from the Android Architecture Components, which can be used to observe changes in the database. By returning a LiveData object from a DAO method, Room automatically updates the returned LiveData whenever there are changes in the database.

Code Example:

```
@Dao
interface UserDao {
    @Query("SELECT * FROM user")
    fun getAllUsers(): LiveData<List<User>>
}
```

```
// Observe the LiveData in an Activity or
// Fragment
userDao.getAllUsers().observe(this) { users ->
// Update UI with the latest user data
}
```

How can you execute complex database queries using Room?

Room supports complex database queries using the `@Query` annotation in DAO methods. You can write custom SQL queries and pass parameters using placeholders.

```
@Dao
interface UserDao {
    @Query("SELECT * FROM user WHERE age > :minAge")
    fun getUsersOlderThan(minAge: Int): List<User>
}
```

What is the purpose of the `@TypeConverter` annotation in Room?

The `@TypeConverter` annotation allows you to define custom type converters in Room. It is useful when you need to store complex data types (such as `Date`, `List`, or custom objects) in the database as primitive types.

What is the purpose of the @TypeConverter annotation in Room?

```
class Converters {  
    @TypeConverter  
    fun fromTimestamp(value: Long): Date {  
        return Date(value)  
    }  
}
```

```
@TypeConverter  
fun dateToTimestamp(date: Date): Long {  
    return date.time  
}  
}
```

```
@Database(entities = [User::class], version = 1)  
@TypeConverters(Converters::class)  
abstract class AppDatabase : RoomDatabase() {  
    // Database definition  
}
```


Exercise



Création d'une Application avec une Base de Données Locale en utilisant ROOM

Mettre en pratique la création d'une base de données locale dans une application Android en utilisant l'ORM Room et implémenter les opérations **CRUD** (**Create, Read, Update, Delete**).

Tâches :

1. Mise en place du projet Android
2. Définir l'Entité
3. Créer le DAO (Data Access Object)
4. Implémentation de la Base de Données
5. Initialisation de la Base de Données
6. Opérations CRUD
7. Utilisation dans l'Application

Thank you !

Questions ? abdelkader.ouared@univ-tiaret.dz

